

Week 2: Product and Category Management System

The store wants to maintain information about products and their categories. Each product belongs to a specific category, and the system should manage product details such as product name, price, and stock quantity.

Design Product and Category Tables

Category Table:

The Category table is created to store the details of different product categories such as Laptops, Cameras, Watches and Bags.

```
SQL> Create table Category(Category_ID varchar(7),Category_name varchar(20),Description varchar(50));  
Table created.
```

Product Table:

The Product table is created to store product details such as Product ID, Product Name, Category ID, Price and Stock Quantity.

```
SQL> Create table Product(Product_ID varchar(7),Product_name varchar(20),Category_ID varchar(7),Price number(9,2),Stock_Quantity number);  
Table created.
```

The two tables for Category and Product have been created. Now, we have to find the relationship between the tables and apply constraints to the tables.

Define Relationships and Constraints

Relationships:

One Category can have many Products. The relationship between Category and Product is **One-to-Many (1 → *)**. The **Category_ID** in the Category table is used to connect the Category table with the Product table.

Primary Key:

A Primary Key is a column or field that uniquely identifies each row or tuple in a table. In the Category table, **Category_ID** uniquely identifies each category. In the Product table, **Product_ID** uniquely identifies each product.

The Category_ID is assigned as the Primary Key for the Category table.

```
SQL> Alter table category add constraint pk_category primary key (category_ID);  
Table altered.
```

The Product_ID is assigned as the Primary Key for the Product table.

```
SQL> Alter table product add constraint pk_product primary key (Product_ID);  
Table altered.
```

Foreign Key:

A Foreign Key is a key that links two tables. Therefore, Category_ID acts as the Foreign Key in the Product table.

The Category_ID in the Product table is connected with Category_ID in the Category table using a Foreign Key.

```
SQL> Alter table product add constraint fk_product_category foreign key (category_ID) references category(category_ID);  
Table altered.
```

NOT NULL:

NOT NULL ensures that a column cannot have an empty or NULL value. **For example**, Category Name and Product Name should not be empty.

Category Name is made NOT NULL.

```
SQL> Alter table category modify category_name varchar(100) NOT NULL;  
Table altered.
```

Product Name is made NOT NULL.

```
SQL> Alter table product modify product_name varchar(150) NOT NULL;  
Table altered.
```

CHECK:

CHECK validates whether the values in a column satisfy a specified condition.
For example, the price of a product must be greater than zero and stock quantity cannot be negative.

The price of a product must be greater than zero.

```
SQL> Alter table product add constraint chk_Price check (price > 0);  
Table altered.
```

The stock quantity must be greater than or equal to zero.

```
SQL> Alter table product add constraint chk_stock check (Stock_Quantity >= 0);  
Table altered.
```

UNIQUE:

UNIQUE ensures that duplicate values are not allowed in a column
For example, two categories cannot have the same category name.

The Category Name is made UNIQUE.

```
SQL> Alter table category add constraint Uq_Category UNIQUE (category_name);  
Table altered.
```

Insert Values

Category Table:

```
SQL> insert into category values ('&category_ID','&category_name','&Description');
Enter value for category_id: 01
Enter value for category_name: Laptops
Enter value for description: Portable computers
old 1: insert into category values ('&category_ID','&category_name','&Description')
new 1: insert into category values ('01','Laptops','Portable computers')

1 row created.

SQL> /
Enter value for category_id: 4064
Enter value for category_name: Cameras
Enter value for description: Digital cameras and accessories
old 1: insert into category values ('&category_ID','&category_name','&Description')
new 1: insert into category values ('4064','Cameras','Digital cameras and accessories')

1 row created.

SQL> /
Enter value for category_id: 4065
Enter value for category_name: watches
Enter value for description: wrist watches
old 1: insert into category values ('&category_ID','&category_name','&Description')
new 1: insert into category values ('4065','watches','wrist watches')

1 row created.

SQL> /
Enter value for category_id: 4066
Enter value for category_name: Bags
Enter value for description: Travel and fashion bags
old 1: insert into category values ('&category_ID','&category_name','&Description')
new 1: insert into category values ('4066','Bags','Travel and fashion bags')

1 row created.
```

After Insertion – Category Table

```
SQL> select * from category;

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----
01
Laptops
Portable computers

4064
Cameras
Digital cameras and accessories

CATEGOR
-----
CATEGORY_NAME
-----
DESCRIPTION
-----

4065
watches
wrist watches

4066
bags
```

Product Table

```
SQL> INSERT INTO Product VALUES('&ProductID','&Product_name','&CategoryID','&Price','&Stock_Quantity');
Enter value for productid: 501
Enter value for product_name: Laptops
Enter value for categoryid: 01
Enter value for price: 55000
Enter value for stock_quantity: 10
old 1: INSERT INTO Product VALUES('&ProductID','&Product_name','&CategoryID','&Price','&Stock_Quantity')
new 1: INSERT INTO Product VALUES('501','Laptops','01',55000,10)

1 row created.

SQL> /
Enter value for productid: 502
Enter value for product_name: Cameras
Enter value for categoryid: 4064
Enter value for price: 35000
Enter value for stock_quantity: 15
old 1: INSERT INTO Product VALUES('&ProductID','&Product_name','&CategoryID','&Price','&Stock_Quantity')
new 1: INSERT INTO Product VALUES('502','Cameras','4064',35000,15)

1 row created.

SQL> /
Enter value for productid: 503
Enter value for product_name: Watches
Enter value for categoryid: 4065
Enter value for price: 5000
Enter value for stock_quantity: 25
old 1: INSERT INTO Product VALUES('&ProductID','&Product_name','&CategoryID','&Price','&Stock_Quantity')
new 1: INSERT INTO Product VALUES('503','Watches','4065',5000,25)

1 row created.
```

```
SQL> /
Enter value for productid: 504
Enter value for product_name: bags
Enter value for categoryid: 4066
Enter value for price: 2500
Enter value for stock_quantity: 30
old 1: INSERT INTO Product VALUES('&ProductID','&Product_name','&CategoryID','&Price','&Stock_Quantity')
new 1: INSERT INTO Product VALUES('504','bags','4066',2500,30)

1 row created.
```

After Insertion –Product Table

```
SQL> SELECT * FROM Product;
```

PRODUCT	PRODUCT_NAME	CATEGOR	PRICE	STOCK_QUANTITY
501	Laptops	01	55000	10
502	Cameras	4064	35000	15
503	Watches	4065	5000	25
504	bags	4066	2500	30

Insert Record

```
SQL> INSERT INTO Product (Product_ID,Product_Name,Category_ID,Price,Stock_Quantity) VALUES ('505','Cameras','4064',35000,15);

1 row created.
```

Update Price

```
SQL> UPDATE Product SET price = 7000 WHERE Product_Name = 'Watches';  
1 row updated.
```

Delete a Product

```
SQL> DELETE FROM Product WHERE Product_Name = 'bags';  
1 row deleted.
```

Generate Category-wise Product Reports

Count Number of Products in Each Category

```
SQL> SELECT c.Category_Name,  
2 COUNT(p.Product_ID)AS Product_Count  
3 FROM Product p  
4 JOIN Category c  
5 ON p.Category_ID = c.Category_ID  
6 GROUP BY c.Category_Name;
```

CATEGORY_NAME

PRODUCT_COUNT

Laptops

1

watches

1

Cameras

2

Find Highest-Priced Product in Each Category

```
SQL> SELECT c.Category_Name, p.Product_Name, p.Price AS Highest_Price  
2 FROM Product p  
3 JOIN Category c ON p.Category_ID = c.Category_ID  
4 WHERE p.Price = (SELECT MAX(p2.Price)  
5 FROM Product p2  
6 WHERE p2.Category_ID = p.Category_ID);
```

CATEGORY_NAME

PRODUCT_NAME HIGHEST_PRICE

Laptops

Laptops

55000

Cameras

Cameras

35000

watches

Watches

7000

CATEGORY_NAME

PRODUCT_NAME HIGHEST_PRICE

Cameras

Cameras

35000

Display Categories Having More Than 3 Products

```
SQL> SELECT c.Category_Name
 2  FROM Product p
 3  JOIN Category c ON p.Category_ID = c.Category_ID
 4  GROUP BY c.Category_Name
 5  HAVING COUNT(p.Product_ID) > 3;

no rows selected
```

Find Average Product Price Category-wise

```
SQL> SELECT c.Category_Name, AVG(p.Price) AS Avg_Price
 2  FROM Product p
 3  JOIN Category c ON p.Category_ID = c.Category_ID
 4  GROUP BY c.Category_Name;
```

CATEGORY_NAME

AVG_PRICE

Laptops
55000

watches
7000

Cameras
35000